

Assignment 4: Implement Berkeley algorithm for clock synchronization.

```
import java.util.*;
class Berkeley {
    public static void synchronize(List<Integer> clocks) {
        int n = clocks.size();
        int sum = 0;
        for (int time : clocks) {
            sum += time;
        }
        int avg = sum / n;
        System.out.println("\nAverage Time: " + avg);
        // Step 3: Calculate adjustments and apply
        System.out.println("\nAdjustments:");
        for (int i = 0; i < n; i++) {
            int current = clocks.get(i);
            int adjustment = avg - current;
            System.out.println("Process " + i + " adjustment = " + adjustment);
            clocks.set(i, current + adjustment);
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of clocks: ");
    int n = sc.nextInt();
    List<Integer> clocks = new ArrayList<>();
    System.out.println("Enter clock times:");
    for (int i = 0; i < n; i++) {
        clocks.add(sc.nextInt());
    }
    System.out.println("\nInitial Clock Times:");
    for (int t : clocks) {
        System.out.println(t);
    }
    // Apply Berkeley Algorithm
    synchronize(clocks);
    System.out.println("\nSynchronized Clock Times:");
    for (int t : clocks) {
        System.out.println(t);
    }
}
}
```

Output

```
C:\Users\chet0\OneDrive\Desktop\LP V\A4>java Berkeley
Enter number of clocks: 3
Enter clock times:
200
50
200
```

Initial Clock Times:

200

50

200

Average Time: 150

Adjustments:

Process 0 adjustment = -50

Process 1 adjustment = 100

Process 2 adjustment = -50

Synchronized Clock Times:

150

150

150

Cmd:

cd Desktop

dir

javac Berkeley.java

dir

Java Berkeley

Assignment 3: To develop a distributed system, to find the minimum number of N elements in an array by distributing N/n elements to n number of processors MPI or OpenMP. Demonstrate by displaying the intermediate minimum element calculated at different processors.

```
#include <stdio.h>
```

```
#include "mpi.h"
```

```
int main(int argc, char* argv[])
```

```
{
```

```
    int rank, size;
```

```
    int num[20]; // N = 20
```

```
    MPI_Init(&argc, &argv);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    // Initialize array
```

```
    for(int i = 0; i < 20; i++)
```

```
        num[i] = i + 1;
```

```
    if(rank == 0)
```

```
    {
```

```
        int s[4];
```

```
        printf("Distribution at rank %d\n", rank);
```

```
        // Send N/n elements (20/4 = 5)
```

```

for(int i = 1; i < 4; i++)
    MPI_Send(&num[i * 5], 5, MPI_INT, i, 1, MPI_COMM_WORLD);

int local_max = num[0];

for(int i = 1; i < 5; i++)
{
    if(num[i] > local_max)
        local_max = num[i];
}

// Receive from other processes
for(int i = 1; i < 4; i++)
{
    MPI_Recv(&s[i], 1, MPI_INT, i, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
}

printf("Local max at rank %d is %d\n", rank, local_max);

int final_max = local_max;

for(int i = 1; i < 4; i++)
{
    if(s[i] > final_max)
        final_max = s[i];
}

printf("Final Maximum = %d\n\n", final_max);
}
else
{
    int k[5];
    MPI_Recv(k, 5, MPI_INT, 0, 1, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    int local_max = k[0];

    for(int i = 1; i < 5; i++)
    {
        if(k[i] > local_max)
            local_max = k[i];
    }

    printf("Local max at rank %d is %d\n", rank, local_max);

    MPI_Send(&local_max, 1, MPI_INT, 0, 1, MPI_COMM_WORLD);
}

MPI_Finalize();
return 0;
}

```

Output:

Distribution at rank 0
Local max at rank 1 is 10
Local max at rank 2 is 15
Local max at rank 3 is 20
Local max at rank 0 is 5
Final Maximum = 20

Installation of OPENMPI

1. Download openmpi-4.1.4.tar.bz2 from <http://www.open-mpi.org> in a folder, say LP5.
2. Goto the terminal (Command prompt)
3. Update using `sudo apt-get update` `sudo apt install gcc` {if not already installed}
4. Goto the directory which contains the downloaded file
5. Extract the files using `tar -jxf openmpi-4.1.4.tar.bz2`
6. The directory openmpi-4.1.4 is created
7. Configure, compile and install by executing the following commands `./configure --prefix=$HOME/opt/openmpi` `make` `make install`
8. Now the openmpi folder is created in 'opt' folder of the Home directory.
9. Now the folder LP5 can be deleted (optional)
10. Update the PATH and LD_LIBRARY_PATH environment variable using `echo "export PATH=$PATH:$HOME/opt/openmpi/bin" >> $HOME/.bashrc` `echo "export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HOME/opt/openmpi/lib" >> $HOME/.bashrc`
11. Compile the program using `mpicc` name of the program
12. Execute the program using `mpirun -np N ./a.out`

Assignment 6: Implement Bully and Ring algorithm for leader election.

Ring.java

```
import java.util.Scanner;

public class Ring {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.println("Enter the number of processes: ");
        int num = in.nextInt();

        Rr[] proc = new Rr[num];
        for (int i = 0; i < num; i++) {
            proc[i] = new Rr();
            proc[i].index = i;
            System.out.println("Enter the ID of process " + (i + 1) + ": ");
            proc[i].id = in.nextInt();
            proc[i].state = "active";
            proc[i].f = 0;
        }
        for (int i = 0; i < num - 1; i++) {
            for (int j = 0; j < num - 1; j++) {
                if (proc[j].id > proc[j + 1].id) {
```

```

        Rr temp = proc[j];
        proc[j] = proc[j + 1];
        proc[j + 1] = temp;
    }
}
}
for (int i = 0; i < num; i++) {
    System.out.print "[" + i + " ] " + proc[i].id + " ";
}
proc[num - 1].state = "inactive";
System.out.println("\nProcess " + proc[num - 1].id + " selected as coordinator");
while (true) {
    System.out.println("\n1. Election\n2. Quit");
    int ch = in.nextInt();

    // Reset flags
    for (int i = 0; i < num; i++) {
        proc[i].f = 0;
    }

    switch (ch) {
        case 1:
            System.out.println("Enter the process number that initializes the election: ");
            int init = in.nextInt();
            int temp2 = init;
            int temp1 = init + 1;
            int i = 0;

            while (temp2 != temp1) {
                if (temp1 == num) {
                    temp1 = 0;
                }
                if ("active".equals(proc[temp1].state) && proc[temp1].f == 0) {
                    System.out.println("Process " + proc[init].id + " sends a message to Process " +
proc[temp1].id);
                    proc[temp1].f = 1;
                    init = temp1;
                    i++;
                }
                temp1++;
            }

            System.out.println("Process " + proc[init].id + " sends a message to Process " + proc[temp1].id);
            int max = -1;

            int m = -1;
            for (int j = 0; j < num; j++) {
                if (proc[j].id > m) {
                    m = proc[j].id;
                }
            }

            System.out.println("Process " + m + " selected as coordinator");

```

```

        for (int k = 0; k < num; k++) {
            if (proc[k].id == m) {
                proc[k].state = "inactive";
            }
        }
        break;
    case 2:
        System.out.println("Program terminated.");
        in.close();
        return;
    default:
        System.out.println("Invalid response.");
        break;
    }
}
}

class Rr {
    public int index; // To store the index of the process
    public int id; // To store the ID/name of the process
    public int f;
    public String state; // Indicates whether the process is active or inactive
}

```

Output

C:\Users\chet0\OneDrive\Desktop\LP V\A6>java Ring

Enter the number of processes:

4

Enter the ID of process 1:

2

Enter the ID of process 2:

3

Enter the ID of process 3:

4

Enter the ID of process 4:

5

[0] 2 [1] 3 [2] 4 [3] 5

Process 5 selected as coordinator

1. Election

2. Quit

1

Enter the process number that initializes the election:

3

Process 5 sends a message to Process 2

Process 2 sends a message to Process 3

Process 3 sends a message to Process 4

Process 4 sends a message to Process 5

Process 5 selected as coordinator

1. Election
2. Quit

```
Cmd:
Ring
cd Desktop
javac -version
javac Ring.java
dir
java Ring
```

Assignment 2 : Develop any distributed application using CORBA to demonstrate object brokering. (Calculator Operations)

Client.java

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import CalcApp.*;
import CalcApp.CalcPackage.DivisionByZero;(import CalcApp.DivisionByZero;)
import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import static java.lang.System.out;

public class CalcClient {
    static Calc calcImpl;
    static BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

    public static void main(String args[]) {

        try {
            ORB orb = ORB.init(args, null);
            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef)
            String name = "Calc";
            calcImpl = CalcHelper.narrow(ncRef.resolve_str(name));
            while (true) {
                out.println("1. Sum");
                out.println("2. Sub");
                out.println("3. Mul");
                out.println("4. Div");
                out.println("5. exit");
                out.println("--");
                out.println("choice: ");

                try {
```

```

        String opt = br.readLine();
        if (opt.equals("5")) {
            break;
        } else if (opt.equals("1")) {
            out.println("a+b= " + calcImpl.sum(getFloat("a"), getFloat("b")));
        } else if (opt.equals("2")) {
            out.println("a-b= " + calcImpl.sub(getFloat("a"), getFloat("b")));
        } else if (opt.equals("3")) {
            out.println("a*b= " + calcImpl.mul(getFloat("a"), getFloat("b")));
        } else if (opt.equals("4")) {
            try {
                out.println("a/b= " + calcImpl.div(getFloat("a"), getFloat("b")));
            } catch (DivisionByZero de) {
                out.println("Division by zero!!!");
            }
        }
    } catch (Exception e) {
        out.println("====");
        out.println("Error with numbers");
        out.println("====");
    }
    out.println("");

}

//calcImpl.shutdown();
} catch (Exception e) {
    System.out.println("ERROR : " + e);
    e.printStackTrace(System.out);
}
}

static float getFloat(String number) throws Exception {
    out.print(number + ": ");
    return Float.parseFloat(br.readLine());
}
}

```

Server.java

```

import CalcApp.*;
import CalcApp.CalcPackage.DivisionByZero;
import CalcApp.DivisionByZero;

import org.omg.CosNaming.*;
import org.omg.CosNaming.NamingContextPackage.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;

import java.util.Properties;

class CalcImpl extends CalcPOA {

```

```

@Override
public float sum(float a, float b) {
    return a + b;
}
@Override
public float div(float a, float b) throws DivisionByZero {
    if (b == 0) {
        throw new CalcApp.CalcPackage.DivisionByZero();
    } else {
        return a / b;
    }
}
@Override
public float mul(float a, float b) {
    return a * b;
}
@Override
public float sub(float a, float b) {
    return a - b;
}
private ORB orb;
public void setORB(ORB orb_val) {
    orb = orb_val;
}
}

public class CalcServer {
    public static void main(String args[]) {
        try {
            ORB orb = ORB.init(args, null);
            POA rootpoa = POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootpoa.the_POAManager().activate();
            CalcImpl helloImpl = new CalcImpl();
            helloImpl.setORB(orb);

            org.omg.CORBA.Object ref = rootpoa.servant_to_reference(helloImpl);
            Calc href = CalcHelper.narrow(ref);

            org.omg.CORBA.Object objRef = orb.resolve_initial_references("NameService");

            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);

            // bind the Object Reference in Naming
            String name = "Calc";
            NameComponent path[] = ncRef.to_name(name);
            ncRef.rebind(path, href);

            System.out.println("Ready..");

            // wait for invocations from clients
            orb.run();
        } catch (Exception e) {

```

```

        System.err.println("ERROR: " + e);
        e.printStackTrace(System.out);
    }

    System.out.println("Exiting ...");

}
}

```

Output

Terminal 1

```

C:\Users\chet8>cd C:\Users\chete\OneDrive\Desktop\LP V\A2
C:\Users\chete\OneDrive\Desktop\LP V\A2>java CalcServer -ORBInitialPort 1058 -ORBInitialHost localhost
Ready..

```

Terminal 2

```

C:\Users\chet@>cd C:\Users\chet@\OneDrive\Desktop\LP V\A2
C:\Users\chet@\OneDrive\Desktop\LP V\A2>java CalcClient -ORBInitialPort 105
1. Sum
2. Sub
3. Mul
4. Div
5. exit
choice:3
a: 569
b: 87
a+b= 49503.8

```

```

1. Sum
2. Sub
3. Mul
4. Div
5. exit
choice:2
a: 89
b: 47
a-b= 42.8

```

```

1 Sum
2. Sub
3. Mul
4. Div
5. exit
choice: 4
a: 360
b: 6
a/b= 60.0

```

```

1. Sum
2. Sub

```

3. Mul
4. Div
5. exit
choice:3
a: 360
b: 6
a*b = 216

Calc.idl

```
module CalcApp {  
  
    exception DivisionByZero {};  
  
    interface Calc {  
  
        float sum(in float a, in float b);  
  
        float sub(in float a, in float b);  
  
        float mul(in float a, in float b);  
  
        float div(in float a, in float b)  
        raises(DivisionByZero);  
    };  
};
```

Calc